CourierLedger M1 Database Design Specification

Technical Correction Amendment v0.5

A targeted correction amendment to the Database Design Specification v0.1 (36 pages), as amended by Technical Correction Amendment v0.2 (CL-01 to CL-12), v0.3 (CL-13 to CL-17) and v0.4 (CL-18 to CL-22). This amendment supersedes only what it explicitly names. Everything else in v0.1 to v0.4 stands unchanged.

> **Scope guard.** This document contains no application code and no executable migrations. SQL fragments, update predicates and configuration settings are illustrative design notation, marked ADAPT, exactly as in v0.1 Section E and v0.2 to v0.4. No courier rate, fee, statement format or contract semantic is invented. V2.2.1 formulas and identities, integer BIGINT poisha, per-component half-up rounding, NULL versus verified zero, and the explicit D_c confirmation-plus-evidence rule are preserved without modification. CL-01 to CL-22 are preserved except where a finding below explicitly supersedes a named detail.

What this amendment does

- Corrects four specific technical gaps, CL-23 to CL-26, each in the established six-part form: original problem, exact change, affected sections and tables, constraint or transaction-level enforcement, updated test scenario, and a resolved / partially resolved / still-open verdict.
- Adds no table, no column and no status; the design total stays at 31 tables. The changes are a removed fixture route (CL-23), two corrected predicates (CL-24), an extended lock protocol (CL-25) and a seventh rule in an existing pattern (CL-26).
- Ends with a consolidated test-change table, an invariant traceability delta, a change log from v0.4 to v0.5, the remaining blockers, and a schema-freeze statement.

Reading rule. Where this amendment and v0.1, v0.2, v0.3 or v0.4 disagree, this amendment wins for the named item only. Where it is silent, v0.1 as amended by v0.2, v0.3 and v0.4 applies.

> **Gating - unchanged.** M1 production implementation remains gated behind the Day 7 and Day 14 market-validation gates, merchant-confirmed courier evidence, a Docker + PostgreSQL verification environment, and explicit production-readiness approval. This amendment authorises none of those steps, and no test in it is claimed passed.

Baseline	Database Design Specification v0.1 (36 pages)
Prior amendments	Technical Correction Amendment v0.2 (CL-01 to CL-12); v0.3 (CL-13 to CL-17); v0.4 (CL-18 to CL-22)
This amendment	v0.5 - CL-23 to CL-26
Financial authority	Specification V2.2.1 and the P0 final clarification, unchanged

Amendment at a glance

Four findings. Each corrects a place where an earlier statement, read precisely, does not say what the design needs it to say: a fixture route whose privilege claim no PostgreSQL role actually satisfies (CL-23), two lifecycle predicates that omit the very condition the same section makes definitional for ownership (CL-24), a lock protocol that covers the creation of a live session but is silent about its renewal (CL-25), and a hardened execution pattern that never fixes the one setting - `search_path` - through which a caller can redirect what a SECURITY DEFINER function resolves (CL-26).

ID	Subject	Affected locations	Verdict
CL-23	CL-18 legacy fixture loading: load-before-trigger becomes the sole permitted route; the replica-mode route and its privilege statement are removed	v0.4 CL-18 sections 2, 4; test T-26(e)	Design-resolved; verification outstanding
CL-24	CL-19 expired-worker fencing: completion and worker-reported failure gain the live-lease condition	v0.4 CL-19 section 2 predicate table; v0.3 CL-14; tests T-27(r), new T-27(u)-(z)	Design-resolved; verification outstanding
CL-25	CL-20 lock protocol extended to session renewal, refresh, expiry extension and reactivation; revocation and deletion position stated	v0.4 CL-20 sections 2, 4; v0.1 Section C, Section G; tests T-30(m)-(p)	Partially resolved - mechanics design-resolved; OD-16, OD-17 still open
CL-26	CL-22 pattern gains Rule 7: restricted `search_path`, trusted-schema discipline, full qualification, no dynamic SQL, explicit EXECUTE revocation	v0.4 CL-22 Rules 1-6; v0.1 Section E; tests T-32(g)-(h)	Design-resolved; verification outstanding

What changes structurally - and what does not

No new tables. The total stays at 31. No finding in this amendment adds a table, a column, an index or a status. CL-23 removes an option rather than adding one. CL-24 changes two rows of an existing predicate table. CL-25 extends an existing protocol's operation list. CL-26 adds one rule to an existing six-rule pattern.

No formula changes. Nothing here touches C, Δ, V_R, S, V_{cond}, V_{total}, the identities $V_{total} = C - \Delta - V_R + S$ and $V_{cond} = S - V_R$, the CL-01 five-step status derivation, the fee-base rules, or the D_c confirmation-plus-evidence requirement.

No decision is silently closed. OD-12, OD-16 and OD-17 stay open, exactly as v0.4 left them. CL-25 touches the mechanics around OD-16 and OD-17 and decides neither.

> **Design resolution is not verification.** A verdict of design-resolved in this amendment means the contradiction or gap is removed and a concrete, PostgreSQL-accurate mechanism is specified. It does not mean the mechanism has been run. No M1 implementation exists, the Docker + PostgreSQL verification environment (blocker B-3) is not established, and every mechanism named here remains unproven against a running database until that environment exists. The final section returns to what this means for schema freeze.

CL-23 - Correct the CL-18 legacy fixture loading mechanism

1. Original problem

v0.4 CL-18 resolved the contradiction between the CL-13 deferred constraint trigger and test T-26(e) by scoping the trigger to the malformation a transaction introduces, and by moving T-26(e)'s two-root history into an explicitly controlled legacy fixture. For loading that fixture, CL-18 section 2 permitted two routes: load-before-trigger (preferred), and a replica-mode load in which the fixture loader, running as the migrator role in a disposable test database, loads the malformed rows with trigger firing suppressed for that loader session - described, in design notation, as "a session-level trigger-suppression setting available only to the table owner".

That privilege statement is not accurate for any PostgreSQL mechanism it could refer to, and the inaccuracy is not cosmetic:

- **The session-level setting is superuser-only.** The setting that suppresses trigger firing for a session (`session_replication_role`) can be set only by a superuser. Table ownership confers no right to set it. A reader implementing CL-18 as written would either fail to run the fixture, or - worse - run the fixture loader as a superuser to make it work, normalising superuser use in the test harness that is supposed to model production discipline.
- **The owner-level mechanism is not a session setting.** What a table owner can do is disable a trigger by DDL on the table. That is not session-scoped: it changes firing for every session, persists until explicitly reversed, and opens a window in which any concurrent writer - not just the loader - commits without the trigger. Describing an owner capability and a session capability as one mechanism merges two different risk profiles.
- **The route is unnecessary.** The load-before-trigger route already achieves everything the fixture needs, with no suppression of any kind: the malformed rows are loaded before the trigger exists, which is also the honest model of how real legacy data comes to exist. The second route adds privilege ambiguity and a restoration step ("restore normal firing before any assertion runs") whose omission would silently invalidate the test, in exchange for nothing.

The production trigger itself is not in question. CL-18's delta scoping, the linear-chain invariant, the partial unique successor index, the composite self-FK and the append-only model all stand. Only the fixture's second loading route, and the privilege claim attached to it, are corrected here.

2. Exact proposed change - one route, stated completely

Supersedes the two-route fixture paragraph in v0.4 CL-18 section 2. The replica-mode route is **removed**, not retained with qualifications. The load-before-trigger route is the sole permitted method for creating a malformed legacy history in a test database, and it is specified as an ordered procedure (design notation, ADAPT):

1. **Disposable database only.** The fixture exists only in a test database built by the test harness and destroyed after the run. It is never loaded into any database that also serves as a development, staging or production environment.
2. **Load before the trigger exists.** The malformed rows - in T-26(e), two roots for one candidate, one chain ending confirmed and one ending rejected - are inserted at the point in the test database's build where the tables and their statement-time constraints exist but the CL-13 deferred constraint trigger has **not yet been created**. No trigger is disabled, dropped or suppressed at any point; there is nothing to restore, because nothing was switched off.
3. **Create the trigger with the production definition.** The CL-13 deferred constraint trigger is then created using exactly the definition the production migration will use - same timing, same delta scoping per CL-18, same execution context per CL-22 as extended by CL-26 of this amendment. A test-only variant of the trigger invalidates the fixture's purpose and is not permitted.
4. **Assert with the trigger fully active.** Every detection assertion - the derivation reporting the candidate as conflicted, the dependent run classifying the shipment `needs_review`, the fixture rows remaining byte-identical - executes after step 3, with the trigger in force.
5. **Prove the boundary in the same run.** The same test database, with the trigger active, must also demonstrate T-26(h): ordinary inserts by the runtime role, and separately by the migrator role in an ordinary session, cannot commit a newly introduced malformation. The fixture is evidence about pre-existing data only if the same database proves new malformation is still rejected.
6. **Label the data.** The fixture rows are identified in the test itself as legacy data loaded by this route. No test output may present them as the product of an application-path insert.

> **What this amendment does not permit.** No role created by this design - runtime, migrator, maintenance or test loader - is granted superuser, and no procedure in this design sets a session-level trigger-suppression setting or disables a constraint trigger, in any database. If a future test requirement genuinely cannot be met by the load-before-trigger route, it returns as a new amendment that names the exact PostgreSQL mechanism, the exact privilege it requires, who holds that privilege, its execution boundary and its restoration procedure. Until such an amendment exists, the route does not exist.

3. Affected sections and tables

v0.4 CL-18 section 2 (the fixture paragraph: the replica-mode route and the sentence describing a session-level suppression setting as available to the table owner are deleted; the six-step procedure above replaces the route description); v0.4 CL-18 section 4 (the fixture-discipline paragraph is narrowed accordingly - see section 4 below); Section I test T-26(e) (revised, section 5 below). Table `shipment_match_decisions`: no column, index or constraint changes. The CL-13 trigger definition, the partial unique successor index, the composite self-FK, the same-row self-reference CHECK and the append-only grant model are unchanged.

4. Constraint or transaction-level enforcement

Production trigger untouched. The deferred constraint trigger keeps the CL-18 delta scope: it rejects malformation the committing transaction introduces, for every role including the migrator, and is neither repaired nor blocked by pre-existing malformation. This finding changes how test data is staged, never what the trigger enforces.

Fixture discipline, narrowed. v0.4 CL-18 section 4 stated that trigger suppression for fixture loading exists only in the test harness, only in the loader session, and only in a disposable database. That sentence is superseded: there is no sanctioned suppression to confine. The enforcement boundary is now the build order of the test database (fixture first, trigger second) plus the CL-11 deployment check, which gains one verification item: the deployed trigger set must match the design, and no production or maintenance procedure may contain a trigger-suppression or trigger-disable step for fixture purposes. Because the permitted route uses an ordinary build sequence, conformance is checkable by inspecting the test harness's build order - a stronger position than auditing that a suppression was switched back on.

Detection remains read-only. As in CL-18, the derivation pronounces on the fixture without modifying it; the malformed rows remain byte-identical throughout, as the append-only model already required.

5. Updated test scenario

T-26(e) (revised again). Pre-existing malformed history, detected read-only. Build the two-root history strictly by the load-before-trigger procedure of section 2: rows inserted before the CL-13 trigger is created, trigger then created from the production definition, all assertions executed with the trigger fully active. Expected: the derivation reports the candidate conflicted with no current decision; a run that would rely on it classifies the shipment `needs_review` with a data-quality finding; neither fixture row has been altered; and the test records the load route used. Any version of this test that suppresses, disables or drops the trigger at any point is non-conforming.

T-26(h) (reaffirmed, unchanged). With the trigger enabled in the same test database, attempts by the runtime role and separately by the migrator role in an ordinary session to commit a second root for a candidate that already has one both fail at commit; the same holds for a further root added to the fixture candidate, whose legacy rows remain as loaded.

6. Status

Design-resolved; implementation verification outstanding. The fixture now has exactly one loading route, that route requires no privilege beyond ordinary test-database construction, and no sentence in the design attributes a session-level suppression capability to table ownership. The deleted route cannot be revived by an implementer without a new amendment. Whether the harness build order is honoured in practice is a B-3 verification item, not a design question.

CL-24 - Expired-worker completion and failure fencing

1. Original problem

v0.3 CL-14 separated the two clocks on `idempotency_keys` - `lease_expires_at`, the ownership horizon, and `expires_at`, the 24-hour replay horizon - and v0.4 CL-19 stated the ownership rule in words and then failed to carry it into two of its own predicates:

- **The words.** CL-19 provides that renewal after expiry is not permitted, and explains why: "an expired-but-unreclaimed lease is not ownership, because a reclaimer may already have read the expired state and be between its read and its conditional update". The renewal predicate therefore requires `lease_expires_at > now()` in addition to status and token.
- **The predicates.** The completion row of the same table requires only `status = 'in_progress'` and `claim_token = :caller_token`, with the justification that "completion carries no lease-time condition of its own: the token is the fence". The worker-reported failure row is identical in shape.

Both cannot be the ownership model. Under the predicates as written, a worker whose lease has expired - and who therefore, by the section's own reasoning, no longer owns the claim - can still complete the operation, or report it failed, at any moment before a reclaimer or the sweeper happens to rotate the token. The window is not theoretical: it is exactly the interval CL-19's renewal reasoning identifies, between expiry and the next transition, and it widens whenever reclaim traffic is light - which is precisely when an abandoned, slow or partitioned worker is most likely to reappear. A completion recorded in that window commits financial work under a claim the design elsewhere says the worker does not hold; a failure recorded in that window discards a claim a reclaimer is about to pick up.

The inconsistency also reaches a test. v0.4 T-27(r) asserts that a worker whose renewal failed after expiry sees its "subsequent completion attempt also affect zero rows **once a reclaim has rotated the token**" - wording that concedes completion succeeds if no reclaim has occurred, contradicting the ownership rule the same amendment states. Section 2 below adopts the recommended rule and removes the contradiction rather than re-describing it: the token fences out a displaced worker, and the lease condition fences out an expired one; both are required, for every worker-initiated transition.

2. Exact proposed change - one ownership rule for every worker transition

Supersedes the completion and worker-reported failure rows of the predicate table in v0.4 CL-19 section 2, and the completion justification sentence that follows that table. The ownership rule for `idempotency_keys` is now uniform:

> A worker owns a claim only while all three hold: the row is `in_progress`, the presented `claim_token` matches, and `lease_expires_at` is later than the database clock at the moment the transition executes. Every worker-initiated transition - renewal, completion, worker-reported failure - requires all three conditions. No worker-initiated transition is exempt.

Operation	Required predicate (all conditions) and effect on success - revised rows only
Completion	`status = 'in_progress' AND claim_token = :caller_token AND lease_expires_at > now()`. Performed as the last write of the work transaction, as CL-14 requires. Effect: `status = 'completed'`, `completed_at` set, the operation's typed result reference set, `lease_expires_at = NULL`. **Zero rows: the worker has lost ownership - the entire work transaction rolls back, discarding all of its results; nothing is partially recorded.**
Worker-reported failure	`status = 'in_progress' AND claim_token = :caller_token AND lease_expires_at > now()`. Effect: `status = 'failed'`, `lease_expires_at = NULL`. **Zero rows: the worker has lost ownership and writes nothing further;** the row's state is whatever the winning transition left. A same-hash retry may re-claim a row failed this way immediately, as CL-14 already allowed.

All other rows of the v0.4 CL-19 predicate table - lease renewal, retry-driven reclaim, cap transition, sweeper failure transition, purge - are unchanged. The common identity conditions of that table (merchant, operation, key and `request_hash` match; a hash mismatch is the CL-04 conflict error in every state) apply to the revised rows exactly as before.

Consequences, stated rather than implied. A worker cannot renew, complete or report failure after its lease expires - the three prohibitions now rest on one rule instead of on a rule for renewal and an exception for the other two. Token rotation on reclaim remains mandatory and unchanged: it is still the mechanism that fences out a worker displaced by an actual reclaim, and the lease condition does not replace it, because a reclaimed row's new lease is live and only the rotated token distinguishes the new owner from the old. Reclaim and sweeper behaviour are unchanged: both already key on the expired lease (`lease_expires_at <= now()`), and this finding gives them no new authority.

The operational duty this creates. Because completion is the last write of a work transaction, a long-running operation must renew its lease - while the lease is still live - so that the lease is unexpired at completion time. A worker that lets its lease lapse mid-operation loses the operation, however much work it has done, and the work rolls back. That is the intended semantics of a lease, and it is why the lease duration remains an operational default (ADAPT, CL-14) to be chosen against real operation durations in the verification environment - not a reason to exempt completion from the condition.

3. Affected sections and tables

v0.4 CL-19 section 2, predicate table (completion and worker-reported failure rows superseded; the paragraph justifying a lease-free completion predicate is deleted); v0.4 CL-19 section 4 (the sentence describing completion as fenced by token alone is read subject to section 2 above); v0.3 CL-14 sections 2 and 4 (any statement that completion or worker-reported failure turns on token match alone is superseded by the uniform ownership rule); Section I test T-27 (part (r) narrowed, parts (u)-(z) added - section 5 below). Table `idempotency_keys`: no schema change. The predicates run entirely on the CL-04 / CL-14 columns. Claim-token fencing, same-transaction completion, atomic reclaim, `attempt_count` and the configured attempt cap (design default 5, ADAPT) are preserved.

4. Constraint or transaction-level enforcement

Single conditional updates, as before. Each transition remains one statement whose predicate is evaluated against the row under PostgreSQL's row lock. Adding the lease condition changes which predicate can be observed true; it introduces no check-then-update sequence and no new locking. All clock comparisons use the database clock at statement execution, never a worker-supplied timestamp, exactly as CL-19 already required - a skewed worker clock can no more complete an expired claim than it could renew one.

Exactly one transition wins each race. A completion racing a reclaim serialises on the row lock: if the completion executes first while its lease is still live, it commits and the reclaim's expired-lease condition is then false against a `completed` row; if the lease has expired before the completion executes, the reclaim (or sweeper) condition is the one that can be true, the token rotates or the row fails, and the completion affects zero rows. There is no interleaving in which both succeed, because both predicates cannot be true of the same locked row state, and the second to execute re-evaluates against the state the first committed. The same argument covers failure racing the sweeper.

Rollback is part of the predicate's meaning. Completion is the last write of the work transaction (CL-14). A zero-row completion is therefore not a status the caller may shrug off: the transaction cannot commit, because committing it would persist financial results with no completed idempotency record - the duplicate-execution hole CL-14 exists to close. The design states this as a rule of the shared idempotency component (v0.1 Section H), not as guidance to individual callers.

State-to-column CHECK unchanged. The CL-14 CHECK (completed and failed rows have `lease_expires_at` NULL; in-progress rows have it NOT NULL with result references NULL) continues to hold after every transition above; each revised predicate's effect maintains exactly that.

5. Updated test scenario

T-27(r) (narrowed). The renewal assertions are retained: renewal before expiry succeeds and extends the lease; renewal attempted after expiry, with no reclaimer yet, affects zero rows. The closing clause - that the worker's subsequent completion affects zero rows once a reclaim has rotated the token - is superseded by part (v) below, which asserts the stronger outcome that no reclaim is needed.

T-27(u)-(z) (new) - Expired-worker fencing. All parts run against the CL-04 / CL-14 / CL-19 lifecycle as amended by this finding, with the database clock as the only time source:

- **(u) Completion before lease expiry.** A worker completes while `status`, token and live lease all hold: the conditional update affects one row, the work transaction commits, and a same-key replay returns the stored result without re-executing (CL-14 replay behaviour, unchanged).
- **(v) Completion after lease expiry, no reclaimer.** The lease expires; no retry, reclaim or sweeper action occurs; the original worker attempts completion with the original token. Expected: zero rows; the work transaction rolls back in full; the key row remains `in_progress` with its original token and expired lease, awaiting reclaim or the sweeper.
- **(w) Worker-reported failure after lease expiry, no reclaimer.** As (v), with a failure report instead of completion. Expected: zero rows; the row is not failed by the expired worker and remains reclaimable; a subsequent same-hash retry reclaims it under a rotated token.
- **(x) Completion racing a reclaimer.** At the expiry boundary, the original worker's completion and a retry-driven reclaim execute concurrently. Expected: exactly one transition wins - either the completion commits (lease still live at execution) or the reclaim rotates the token and the completion affects zero rows and rolls back. The operation's financial effects exist at most once.
- **(y) Failure racing a sweeper.** The expired worker's failure report and the sweeper transition execute concurrently. Expected: exactly one wins; the row ends `failed` by one route only, with `lease_expires_at` NULL, and the losing statement affects zero rows and writes nothing.
- **(z) Rollback on zero-row completion.** Force the (v) outcome inside a work transaction that has already written operation results in earlier statements. Expected: on rollback, none of those writes persists; no run, batch or result row references a completion that was never recorded.

6. Status

Design-resolved; implementation verification outstanding. CL-14 and CL-19 now state one ownership model, and every worker-initiated predicate expresses it. The alternative - a token-only model in which expiry ends nothing until a transition occurs - was not adopted: it would leave the expired-but-unreclaimed window governed by whichever process happened to run next, which is the defect CL-19's renewal rule already recognised. The race outcomes in T-27(x) and (y) are argued from row-lock serialisation and must be demonstrated under concurrent load in the B-3 environment before this finding is treated as verified.

CL-25 - Session renewal and the shared-user lock protocol

1. Original problem

v0.4 CL-20 replaced CL-15's membership-row locking with a uniform protocol: a transaction-scoped advisory lock keyed by `user_id`, acquired before reading or writing protected state, shared by membership creation, invitation creation and acceptance, reactivation, eligibility-affecting status changes, session creation, and workspace Stage 2 anonymisation. Its Stage 2 recheck asks three questions under the lock, one of which is whether a live session exists for the user - no `user_sessions` row that is unrevoked and unexpired.

The protocol's operation list, however, names only one session operation: creation. It is silent about every operation that can make an existing session live again or keep it live longer:

- **Renewal or refresh** - a refresh operation that extends a session's validity, whether or not it also rotates any credential material.
- **Expiry extension** - any update that moves a session's expiry later, including sliding-expiry behaviour on ordinary authenticated activity, if the design supports it.
- **Reactivation** - any operation that returns an expired session to live status, if supported.

The hole is the CL-20 hole in miniature. CL-20 existed because a lock taken only over existing rows cannot stop the insert racing it. Here, session creation takes the lock, but a renewal transaction that does not would not wait for Stage 2 and would not be waited for by it: Stage 2 could recheck, correctly find no live session, and anonymise a user whose session a concurrent refresh has just extended - leaving a live session attached to an anonymised identity, and a refresh path by which that identity continues to act. CL-20's recheck is only as sound as the completeness of the list of operations that take the lock, and the list was incomplete on exactly the side the recheck reads.

Two related silences are corrected with it. CL-20 never stated whether session revocation and deletion participate in the protocol, leaving an implementer to guess in both directions. And it never stated what follows if an implementation offers no renewal operation at all - whether the gap is then closed by absence, and what in the schema would make that absence real rather than accidental.

2. Exact proposed change - the protocol covers liveness, not just creation

Extends the operation list in v0.4 CL-20 section 2. The rule is stated by effect, so that no naming variation escapes it:

> Every transaction whose effect can be that a live session exists for a user after it commits, where none need have existed before, must acquire the CL-20 transaction-scoped per-user advisory lock for that `user_id` before it reads or modifies the relevant session or membership state, and must hold it to commit.

Operation class	Lock?	Reason
Session creation (sign-in, invitation acceptance that signs in, any first issuance)	Required	Already in CL-20; restated so the full list lives in one place.
Session renewal or refresh, including any refresh that rotates session material and issues a replacement	Required	Its effect is a live session after commit. A rotation that revokes the old row and creates a new one is a creation-class operation, not a revocation.
Any operation that extends session expiry, including sliding expiry applied on activity	Required	Extension is renewal by effect, whatever the endpoint is called.
Reactivation of an expired session, if supported	Required	It converts an ineligible-blocking state (no live session) into its opposite under exactly the condition Stage 2 tests.
Pure session revocation (logout, administrative revoke) that creates and extends nothing	Not required	Stated explicitly in section 2 (continued) below.
Session deletion or expiry sweeping that creates and extends nothing	Not required	Same reasoning as revocation.

2. Exact proposed change (continued) - revocation, deletion, and Stage 2

Why revocation and deletion stand outside the lock. Revocation and deletion can only move a session away from liveness; neither can create, extend or restore one. The worst outcome of either racing Stage 2 is therefore conservative in the safe direction: Stage 2's recheck, reading committed state under its lock, may see a session that a concurrent revocation is about to end and defer an anonymisation that could have proceeded. A deferred anonymisation is a safe, retryable outcome; the protocol reserves its locking duty for operations that can produce the unsafe outcome - a live session for an anonymised user - and those are the creation-class operations above. This is a deliberate asymmetry, stated so that no implementer "completes" the protocol by locking revocation (harmless but unnecessary contention on every logout) or, worse, reasons in reverse that because revocation needs no lock, renewal needs none either.

Stage 2, unchanged in form, now sound in substance. Stage 2 continues to acquire the per-user lock for each candidate in ascending lock-key order and to recheck eligibility under the lock, including the live-session condition exactly as CL-20 states it: no `user_sessions` row for the user that is unrevoked and unexpired. With renewal, refresh, extension and reactivation inside the protocol, that recheck is no longer defeasible by a session operation: any transaction that could make a session live either committed before the recheck (and is visible to it) or waits until Stage 2 commits, at which point it reads the anonymised `users` row and the application's own rules decide whether a session may be issued to it - with the database-level guarantee, asserted by test T-30(p) below, that the anonymisation is never the outcome that loses the race silently.

If renewal is impossible by design. This amendment does not assert that the product will or will not offer session renewal; that is an application decision the database design does not need. What it removes is the ambiguous middle state. An implementation that offers no renewal must make that a stated design constraint, not an accident of the first version: no operation in the session lifecycle may update a session's expiry to a later value or clear its revoked or expired state, and the session component (v0.1 Section H) records the constraint alongside the protocol list, so a future refresh feature is visibly a protocol change - it joins the lock list in the same change that introduces it - rather than a quiet addition that reopens this finding. Conversely, an implementation that does offer renewal needs no schema support beyond the session row it already maintains: the constraint this finding adds is the lock, not a column.

3. Affected sections and tables

v0.4 CL-20 section 2 (the protocol's operation list gains the renewal, refresh, expiry-extension and reactivation classes, and the revocation / deletion position); v0.4 CL-20 section 4 (the Stage 2 recheck paragraph is read with the completed list); v0.1 Section C, table `user_sessions` (access protocol only); Section G (Stage 2 procedure note); Section H (session component records the lock list and, if applicable, the no-renewal constraint); Section I tests T-30(m)-(p) (new, section 5 below), alongside the CL-20 extensions T-25 and T-30(h)-(l), which are unchanged. No table, column or constraint changes. No privilege changes: advisory locks require no table privilege and confer none, as CL-20 already established.

4. Constraint or transaction-level enforcement

Protocol enforcement, honestly labelled - as in CL-20. Cross-table eligibility cannot be a CHECK (CL-17), and no trigger can compel a transaction to have taken an advisory lock it never requested. Enforcement is the protocol itself: one shared session component through which every creation-class session operation passes, stated as a design invariant in the CL-11 / CL-13 / CL-20 sense, with conformance demonstrated by the concurrency scenarios in section 5 rather than assumed. The lock key, namespace, transaction scoping and ascending-order discipline are exactly CL-20's; this finding adds operations to the protocol, not a second protocol.

Stage 2 still fails closed. If any per-user lock cannot be acquired within the maintenance transaction's bounded wait, Stage 2 aborts for that workspace rather than proceeding on a partial lock set - unchanged from CL-20, and now covering the case where the contending holder is a renewal transaction.

What is preserved. The tombstone merchant approach is unchanged: the `merchants` row is anonymised in place, `anonymised_at` is set, no `merchant_id` on any financial record is ever rewritten. A user with an active or invited membership in any other workspace is never anonymised. Stage 2 remains a maintenance transaction under the migrator role only. OD-16 and OD-17 remain open: this finding, like CL-20, strengthens mechanics only and decides neither retention periods nor the treatment of personal data inside append-only raw records.

5. Updated test scenario

T-30(m)-(p) (new) - Session lifecycle racing Stage 2. Each part begins Stage 2 anonymisation for workspace A, where user U is a candidate whose memberships are ended and who has no live session at setup, and runs the named session operation for U concurrently. The asserted pattern is CL-20's either/or, extended to the session side:

- **(m) Renewal racing Stage 2.** U holds an expired session; a renewal that would return it to live status runs concurrently with Stage 2. Expected: the two transactions serialise on U's per-user lock - either the renewal commits first and Stage 2's recheck finds a live session and defers anonymisation, or Stage 2 commits first and the renewal then proceeds, if at all, against an already-anonymised user under the application's own rules. Never: an anonymised user with a session made live by the racing renewal.
- **(n) Creation racing Stage 2.** A fresh sign-in creating a session for U runs concurrently with Stage 2. Expected: the same either/or outcome as (m). This part re-asserts CL-20's session-creation coverage inside the completed protocol, so the creation and renewal cases are proven by one scenario family rather than assumed equivalent.
- **(o) A session becoming live through refresh.** For every refresh-shaped operation the implementation supports - expiry extension, sliding expiry on activity, rotation issuing a replacement, reactivation of an expired session - repeat (m) using that operation. Expected: identical serialisation and identical either/or outcome for each shape. An implementation supporting none of them records that fact against the no-renewal constraint of section 2 instead, and this part is satisfied by the constraint's presence in the session component, not by a vacuous pass.
- **(p) No live session for an anonymised user through a stale refresh.** After Stage 2 has anonymised U, present a refresh request carrying session material issued before the anonymisation, including one that was in flight while Stage 2 ran. Expected: U does not obtain a live session; the request fails or is refused under the application's rules, and no conforming database path exists by which it could have committed during the race.

As in CL-20, parts (m) to (p) are also run under repeated concurrent load with the T-30(l) expectation: PostgreSQL deadlock detection may abort a participant, which the protocols treat as a retryable failure, never as a silent pass.

6. Status

Partially resolved - mechanics design-resolved; policy still open. This mirrors CL-20's verdict deliberately: the serialisation gap on the session side is closed at design level by defining the protocol over session liveness rather than over one named operation, and the revocation / deletion position is no longer left to inference. OD-16 (grace window, retention periods, erasure-versus-retention conflict rule) and OD-17 (third-party personal data in append-only raw records) remain open for Bangladesh legal and policy review, exactly as before, and the protocol's behaviour under real concurrency - now including refresh traffic - is unverified until the B-3 environment exists.

CL-26 - SECURITY DEFINER search_path hardening

1. Original problem

v0.4 CL-22 specified the validation trigger context pattern in six rules: migrator-owned SECURITY DEFINER functions without BYPASSRLS (Rule 1); `merchant_id` derived from the firing rows, never from caller context (Rule 2); an owner-only validation SELECT policy keyed to a transaction-local setting (Rule 3); explicit tenant predicates on every query (Rule 4); a fail-closed anchor check (Rule 5); and no privilege reaching the runtime role (Rule 6). The pattern constrains whose privileges the function executes with, which rows its policies admit, and what it does when it cannot see its evidence. It never constrains **which objects the function's unqualified names resolve to**.

That omission matters because of how a SECURITY DEFINER function resolves names. The function executes with its owner's privileges, but unless the function itself fixes a `search_path`, name resolution follows the path in effect for the session that caused the trigger to fire - in this design, an application session under the runtime role's control. A schema that an untrusted role can create objects in, placed earlier on that path than the schema holding the real validation tables and helper objects, can shadow them: a same-named table, view or function is resolved instead of the intended one. For an ordinary function the consequence is wrong results. For a validation function the consequence is worse and quieter - validation that reads attacker-shaped rows, or calls an attacker-shaped helper, and passes. Rules 3 to 5 do not close this: explicit tenant predicates (Rule 4) constrain rows, not object identity, and the anchor check (Rule 5) proves the function can see **a** row through its queries, not that the object it queried is the table the design means.

A second, smaller imprecision travels with it. Rule 6 states that EXECUTE on validation functions "is not granted to the runtime role". In PostgreSQL, however, EXECUTE is granted to PUBLIC by default at function creation; a function from which nothing was explicitly revoked is callable by every role, trigger-type return or not. "Not granted" is therefore not a state an implementer can rely on - the revocation must be an explicit step, or Rule 6's defence in depth does not exist. Section 2 adds the missing rule and corrects the revocation wording inside the same finding, since both concern how the function is reached and resolved rather than what it validates.

2. Exact proposed change - Rule 7 joins the CL-22 pattern

Adds a seventh rule to the validation trigger context pattern in v0.4 CL-22 section 2. Rules 1 to 6 are retained in full - owner and execution mode, merchant derivation from firing rows, owner-only validation policy, explicit tenant predicates, fail-closed anchor check, no general-purpose RLS bypass - and Rule 7 is mandatory for every SECURITY DEFINER validation function in the design, on the same terms (design notation throughout, ADAPT):

Rule 7 - The function carries its own restricted search_path, and its objects are qualified.

- **(a) Fixed at the function, not the session.** Every validation function is created or altered with its own `search_path` setting attached to the function definition, so the path that governs its name resolution is the one recorded with the function, whoever fires the trigger and whatever path the calling session has set. A session-level or role-level path is not a substitute and is not relied on.
- **(b) Trusted schemas only, by one principle.** The path names only schemas that satisfy a single test: **the migrator / validation-owner role controls object creation in the schema, and no other role - including PUBLIC - holds CREATE on it.** In practice this is the application schema holding the validated tables and the validation functions, together with the system catalog schema, which PostgreSQL resolves first in any case and which no application role can write to. Any schema failing the test - a general-purpose schema, a per-user schema, any schema with a standing CREATE grant to an application or untrusted role - must not appear in the path. The principle, not the schema names, is the invariant: an implementation with differently named schemas applies the test, not this paragraph's examples.
- **(c) Temporary objects cannot shadow.** Because a session can create temporary objects under names matching real ones, the function's fixed path and rule (d) together are the defence: resolution inside the function never depends on the calling session's temporary objects, and a reviewer must be able to confirm that no unqualified reference in the function body could bind to a session-created object.
- **(d) Schema qualification as the second line.** Every reference to an application or validation object inside a validation function - every table, view, sequence and helper function - is written schema-qualified, so that even a path misconfiguration has nothing unqualified to redirect. An unqualified application-object reference in a validation function body is a review defect under the same drafting discipline CL-22 Rule 4 applies to missing tenant predicates.
- **(e) No dynamic SQL.** Validation functions contain no dynamically constructed statements. Validation needs are fixed queries against known tables, so the prohibition costs nothing; it removes the one channel through which parameterised-looking code can still assemble an unqualified or caller-influenced object name. If a future validation genuinely cannot be expressed without dynamic SQL, it returns as an amendment; the exception is not available by implementer discretion. Any such future exception would require, at minimum, schema-qualified object names and strict identifier quoting of every interpolated name - stated here so the floor for that argument is known in advance.

(f) Ownership and RLS posture unchanged. Rule 7 alters neither: each function remains owned by the non-superuser migrator role, which does not carry BYPASSRLS (Rule 1), and every validated table keeps RLS ENABLE with FORCE in force. A fixed `search_path` is a name-resolution control; it grants no row access, and the owner-only validation policy of Rule 3 remains the only route by which the function's reads are admitted.

(g) The runtime role gains nothing - and the default grant is revoked. Corrects the wording of CL-22 Rule 6. The runtime role receives no additional grant, no role membership and no direct execution permission on any validation function. Because PostgreSQL grants EXECUTE to PUBLIC when a function is created, conformance requires an **explicit revocation** of that default grant for every validation function, recorded as a step in the migration pattern (ADAPT) rather than left as an assumption about what was never granted. Direct invocation remains defence in depth - a trigger function cannot usefully be called outside trigger context - but the design now states the revocation instead of implying it.

3. Affected sections and tables

v0.4 CL-22 section 2 (new Rule 7; Rule 6's EXECUTE wording corrected by (g) above); v0.1 Section E (the validation trigger context pattern gains the `search_path`, trusted-schema and revocation requirements, and the CL-11 deployment check gains the corresponding verification items in section 4); Section I test T-32(g)-(h) (new, section 5 below). The pattern applies to every deferred constraint-trigger function in the design, as CL-22 already provided: the CL-01 status trigger, the CL-13 chain trigger as scoped by CL-18 and staged by CL-23, and the CL-16 variance trigger. No table, column or constraint changes. Tenant isolation is not weakened: Rule 7 narrows what a privileged function can resolve to; it widens no role's access to anything.

4. Constraint or transaction-level enforcement

Pattern conformance remains a review gate, now with a catalog check. As CL-22 established, implementation review checks each validation function against the rules in order; Rule 7 adds two mechanically checkable items to that review: the function's recorded configuration carries the restricted `search_path`, and its body contains no unqualified application-object reference and no dynamic statement construction. The first of these is verifiable from the system catalog at deployment time, and the CL-11 deployment check gains it explicitly: for every validation function, the deployed database must show the function-level path setting, the explicit EXECUTE revocation, and the absence of CREATE grants to untrusted roles (including PUBLIC) on every schema in that path. A deployment failing any of the three is non-conforming before any test runs.

Failure mode is refusal, not redirection. If a trusted schema is misconfigured so that the function's qualified references cannot resolve, the function errors and the firing transaction aborts - the same fail-closed posture as Rule 5, extended from rows to objects. Under no configuration does a validation function fall back to resolving an object from a caller-controlled schema and proceed.

5. Updated test scenario

T-32(g)-(h) (new) - Object resolution under attack. Executed, like the rest of T-32, as the runtime role against a database with RLS ENABLE + FORCE and the production policy and grant set - explicitly not as the migrator or a superuser:

- **(g) Shadow object.** From a runtime-role session, set a session `search_path` that places a schema the runtime role controls ahead of the application schema, and create in that schema a shadow object bearing the name of an object a validation function uses - a same-named table holding rows shaped to make an inconsistent transaction appear consistent (and, in a second attempt, a temporary object of the same name). Then commit the T-32(a) transaction: a result whose findings contradict its stored overall status. Expected: the deferred validation still rejects at commit; the function resolved only its intended, schema-qualified objects in its trusted schemas; the shadow object was neither read as evidence nor executed; and a subsequent consistent transaction (T-32(d)) still commits, proving the attack did not merely break validation into refusing everything.
- **(h) Trusted-schema integrity.** As the runtime role, attempt to create any object in a schema named in a validation function's `search_path`: refused, for want of CREATE. Inspect the catalog as the maintenance role: every validation function shows its function-level `search_path` setting restricted to trusted schemas, and no validation function shows an EXECUTE grant reachable by the runtime role, whether directly or through PUBLIC. Any function failing either catalog assertion is a deployment defect under section 4, independent of the behavioural result in (g).

T-32(a) to (f) are unchanged, including the fail-closed assertion (f): disabling the validation policy must produce a Rule 5 refusal, and Rule 7 does not alter that expectation - a function that cannot see its anchor rows raises, whatever its path.

6. Status

> **Design-resolved; implementation verification outstanding.** The CL-22 pattern now constrains privileges, policies, predicates, failure behaviour and name resolution - the five ways a caller could otherwise influence what a validation function does. The shadow-object scenario is, like the rest of T-32, exactly the kind of behaviour that must be demonstrated against the actual PostgreSQL version, roles, grants and RLS configuration in the B-3 environment; SECURITY DEFINER resolution is specified here, not proven, and this finding is not treated as verified until T-32(g) and (h) have run there.

Consolidated test changes

Section I of v0.1, as amended by v0.2, v0.3 and v0.4, stands except as listed here. This table is the complete delta from this amendment. No test is claimed to have passed: these are design-stage scenarios for the future M1 test suite, and the concurrency, fixture and RLS scenarios among them can only be executed once the B-3 verification environment exists.

Test	Change	Effect of this amendment
T-26(e)	Revised (CL-23)	Malformed two-root history is built solely by the load-before-trigger procedure: rows loaded before the CL-13 trigger is created, trigger then created from the production definition, all detection assertions run with the trigger fully active. Any suppression, disabling or dropping of the trigger makes the test non-conforming.
T-26(h)	Reaffirmed	Unchanged. In the same fixture database, ordinary inserts by the runtime role and by the migrator role cannot commit a newly introduced malformation.
T-27(r)	Narrowed (CL-24)	Renewal assertions retained. The closing clause, under which completion by an expired worker failed only after a reclaim rotated the token, is deleted and replaced by T-27(v).
T-27(u)-(z)	New (CL-24)	(u) completion before expiry commits and replays; (v) completion after expiry with no reclaimer affects zero rows and rolls back; (w) worker-reported failure after expiry likewise; (x) completion racing a reclaimer - exactly one transition wins; (y) failure racing a sweeper - exactly one wins; (z) zero-row completion rolls back the whole work transaction, leaving no orphan results.
T-30(m)-(p)	New (CL-25)	(m) renewal racing Stage 2; (n) session creation racing Stage 2; (o) a session becoming live through each supported refresh shape - or, where none is supported, the recorded no-renewal constraint; (p) an anonymised user cannot obtain a live session through a stale or in-flight refresh.
T-32(g)-(h)	New (CL-26)	(g) shadow-object attack via a caller-controlled schema and via a temporary object: validation still resolves only its trusted, qualified objects and still rejects the inconsistent commit; (h) CREATE in a trusted schema refused to the runtime role, and catalog assertions for the function-level `search_path` and the explicit EXECUTE revocation.

Tests T-01 to T-25, T-26(a)-(d), (f), (g), T-27(a)-(q), (s), (t), T-28, T-29, T-30(a)-(l), T-31 and T-32(a)-(f) are unchanged by this amendment.

Invariant traceability delta

The appendix invariant traceability of v0.1, as amended, stands except for the rows below. Each row names the invariant an earlier amendment established, the v0.5 finding that touches it, and the precise sense in which it is preserved, narrowed or strengthened. No invariant is weakened anywhere in this amendment.

Invariant (source)	Finding	Delta
CL-13 linear decision chain; trigger rejects malformed histories (v0.3 CL-13, v0.4 CL-18)	CL-23	**Preserved.** Trigger definition, delta scope, chain invariant and detection derivation unchanged. Only the staging of test data changes: one fixture route instead of two.
Append-only match decision history (v0.2 CL-03 / CL-07)	CL-23	**Preserved.** The fixture is loaded data in a disposable database; no production or application path gains any ability to write outside the append-only model.
Idempotency: no duplicate financial operations; lease separate from replay horizon; claim-token fencing; attempt cap (v0.2 CL-04, v0.3 CL-14, v0.4 CL-19)	CL-24	**Strengthened.** Ownership now requires a live lease for every worker-initiated transition, not renewal alone. Token rotation, atomic reclaim, same-transaction completion, the 24-hour horizon and the cap are unchanged.
Purge never touches a live claim (v0.4 CL-19)	CL-24	**Preserved.** Purge eligibility, the sweeper and reclaim predicates, and the maintenance-only DELETE grant are untouched by this amendment.
Tombstone merchant; shared user never anonymised with a live membership elsewhere; Stage 2 rechecks under lock (v0.3 CL-15, v0.4 CL-20)	CL-25	**Strengthened.** The recheck's live-session condition is now backed by a protocol covering every operation that can produce a live session. `merchant_id` immutability and the tombstone approach are unchanged.
Tenant isolation under ENABLE + FORCE RLS; no general-purpose bypass (v0.1 Section E, v0.2 CL-11, v0.4 CL-22)	CL-26	**Strengthened.** CL-22 Rules 1-6 retained in full; Rule 7 fixes name resolution inside the privileged functions the pattern already relies on, and Rule 6's EXECUTE position is made real by explicit revocation.
V2.2.1 financial semantics: formulas, identities, BIGINT poisha, per-component half-up rounding, NULL versus verified zero, D_c confirmation plus evidence (V2.2.1; v0.2 CL-01, CL-02)	All	**Preserved without modification.** No finding in this amendment touches a formula, a fee-base rule, a rounding rule or a status derivation.
31-table design (v0.1 as amended to v0.4)	All	**Preserved.** No table, column, index or status added or removed by v0.5.

Change log - v0.4 to v0.5

Vocabulary as in v0.4: **Corrects** - the earlier text was wrong or internally inconsistent and is replaced. **Clarifies** - the earlier text was ambiguous or silent; one reading is fixed. **Adds** - new material with no direct counterpart. **Strengthens** - an existing mechanism is replaced or extended by a strictly stronger one for a demonstrated reason. **Removes** - material deleted, with the reason recorded in its finding.

Finding	Kind	Change
CL-23	Removes / Corrects	The replica-mode fixture route in v0.4 CL-18 section 2 is removed. The sentence describing a session-level trigger-suppression setting as available only to the table owner is corrected: the session-level setting is superuser-only, and the owner-level mechanism (trigger-disable DDL) is not session-scoped. Load-before-trigger is specified as a six-step procedure and made the sole permitted route; CL-18 section 4's fixture-discipline sentence is narrowed; T-26(e) is revised; the CL-11 deployment check gains a trigger-set verification item.
CL-24	Corrects / Strengthens	The completion and worker-reported failure rows of the v0.4 CL-19 predicate table gain the condition `lease_expires_at > now()`. The paragraph justifying a lease-free completion predicate is deleted. A uniform ownership rule for all worker-initiated transitions is stated. T-27(r) is narrowed and T-27(u)-(z) are added, covering the six required cases including both races and full rollback.
CL-25	Clarifies / Strengthens	The v0.4 CL-20 protocol's operation list is extended by effect: renewal or refresh, expiry extension including sliding expiry, reactivation of an expired session, and rotation issuing a replacement are all creation-class operations requiring the per-user lock. Pure revocation and deletion are explicitly placed outside the lock, with the reasoning recorded. The position for implementations offering no renewal is stated as a recorded design constraint. T-30(m)-(p) are added.
CL-26	Adds / Corrects	Rule 7 is added to the CL-22 pattern: function-level restricted `search_path` naming only trusted schemas under a single CREATE-control principle, protection against temporary-object shadowing, full schema qualification of application and validation objects, and a prohibition on dynamic SQL in validation functions. CL-22 Rule 6 is corrected so that removal of the runtime role's execution reach is an explicit revocation of the default PUBLIC grant. T-32(g)-(h) are added, and the CL-11 deployment check gains the catalog verification items.

No other change is made by this amendment. In particular, nothing in v0.5 reopens CL-01 to CL-22 beyond the named details above, alters any open decision, or moves any gate.

Remaining blockers and open decisions

Blocking any M1 implementation (unchanged from v0.1, v0.2, v0.3 and v0.4)

- **B-1. Market validation:** Day 7 and Day 14 gates not yet run; no interview evidence exists.
- **B-2. Merchant-confirmed courier evidence:** none exists for any of Steadfast, Pathao, Paperfly, RedX or eCourier - no rule version can yet be populated without inventing contract semantics, which is prohibited.
- **B-3. Docker + PostgreSQL verification environment not established.** No design SQL or mechanism in v0.1 to v0.5 has been executed. This amendment adds four items to that unverified list, and names them so they are tested first when the environment exists: the load-before-trigger fixture build order and T-26(h) boundary (CL-23); the expired-lease race behaviour of the revised completion and failure predicates, T-27(x), (y) and (z) (CL-24); advisory-lock serialisation under concurrent renewal and refresh traffic against Stage 2, T-30(m)-(p) (CL-25); and SECURITY DEFINER name resolution under a caller-controlled path, T-32(g) and (h) (CL-26). These join the five mechanisms v0.4 already listed under B-3, none of which this amendment re-verifies.
- **B-4. Explicit production-readiness approval not given.**

Open decisions carried forward

- **OD-12 (CL-02, v0.2)** - whether any real courier statement reports a negative net payout. Open until redacted merchant statements answer it. Untouched by this amendment.
- **OD-16 (CL-12, v0.2)** - workspace grace-window length, Bangladesh financial retention period, and the erasure-versus-retention conflict rule. Requires policy and legal review. CL-25 supplies session-side serialisation mechanics; the periods and the conflict rule remain undecided. Blocks production launch, not design review.
- **OD-17 (CL-15, v0.3)** - treatment of third-party personal data held inside append-only raw records when a workspace is anonymised. Interim position unchanged: sealed retention, excluded from anonymised exports, no runtime-role access. Requires the same Bangladesh legal review as OD-16. Blocks production launch, not design review.
- **OD-11 (v0.1, narrowed by CL-17)** - guaranteeing at least one active owner per workspace: mechanism choice remains open. Untouched by this amendment.
- **CL-11 deployment check (v0.2)** - the tenant-context trust boundary is specified; verifying a real deployment honours it remains a production-readiness task. This amendment adds three items to that check: the deployed trigger set matches the design with no fixture-suppression step in any production or maintenance procedure (CL-23); no production path depends on a token-only completion or failure transition (CL-24); and every validation function shows the Rule 7 path setting, the explicit EXECUTE revocation, and CREATE-free trusted schemas (CL-26).

All other v0.1 open decisions OD-01 to OD-15 stand as written, except OD-11 and OD-12 as noted above.

Is the database design ready for schema freeze?

Ready to freeze as a design review baseline, as corrected by this amendment - with the three-way distinction this series has maintained kept explicit.

Design freeze means the text stops moving except by targeted amendment on a demonstrated contradiction. v0.4 reached that position and this amendment does not disturb it: the four findings here were precision defects inside mechanisms the freeze already contained, not missing mechanisms. One route was deleted because its privilege claim was wrong (CL-23); two predicates were brought into line with the ownership rule their own section stated (CL-24); a protocol's operation list was completed on the side its recheck reads (CL-25); and a pattern gained the name-resolution rule it had always implicitly assumed (CL-26). No finding required a new table, a formula change, a new status or a reopened decision. Freezing the schema design at v0.1 as amended by v0.2, v0.3, v0.4 and v0.5 - 31 tables, the CL-01 status model, the CL-13 to CL-16 histories, the CL-18 to CL-22 mechanisms, and the CL-23 to CL-26 corrections - is therefore recommended, in the same sense as v0.4 stated it: any further change requires a new targeted amendment with a demonstrated contradiction.

Freeze is not implementation verification. Every mechanism listed under B-3 remains unexecuted, and v0.5 adds four to that list rather than retiring any. A described predicate, lock protocol or path setting is a claim about PostgreSQL behaviour, and several of these claims - the race outcomes in T-27(x) and (y), the serialisation in T-30(m)-(p), the resolution behaviour in T-32(g) - are claims only a running database under the actual roles, grants, triggers and RLS configuration can settle. If the verification environment contradicts one of them, the freeze is reopened for that mechanism by a further amendment and the contradiction is recorded, exactly as v0.4 provided; it is not worked around in code.

Freeze is not implementation authorisation. Market validation, merchant-confirmed evidence and explicit production-readiness approval are all still outstanding (B-1, B-2, B-4). M1 production implementation remains gated behind them, and nothing in this amendment - including the word "freeze" - authorises a migration, a schema, a line of application code or a deployment.

> **Final statement.** With this amendment applied, the specification no longer contains a fixture route whose stated privilege no role holds (CL-23), an ownership rule that its completion and failure predicates contradict (CL-24), a lock protocol whose recheck can be defeated by renewing the thing it checks for (CL-25), or a privileged validation function whose object resolution a caller can redirect (CL-26). Each resolution is at design level only, and this amendment says so in each finding rather than letting a described mechanism read as a proven one. The design is ready for schema freeze as a review baseline. It is not verified, and it is not an authorisation to build.

End of amendment v0.5. This amendment supersedes only the sections it explicitly names. All other content of the Database Design Specification v0.1 and of Technical Correction Amendments v0.2, v0.3 and v0.4 remains unchanged.